

AsymptoticAnalysis after the third review wave

An incremental audit of symbolic contracts, numerical bridges, and Mathics portability

Repository	VladimirReshetnikov/Asymptotic
Reviewed commit	513917b5b387152256b14ac76d92dd30cd13a11d
Review date	September 9, 2026, America/Los_Angeles
Baseline corpus	27 earlier review packages; 167 attributed finding entries before overlap consolidation
Execution here	27 independent mathematical/reference tests and 8 synthetic tooling tests passed
Not executed here	AsymptoticAnalysis in either Mathics or an official Wolfram kernel; all supplied kernel probes remain unrun

Central assessment. The repository has a substantial, differentiated asymptotic algebra and a serious, isolated Mathics compatibility effort. Its mathematical representation is more ambitious than a thin wrapper around native series. The remaining portability boundary is not simply a list of missing built-ins: it includes argument-order semantics, the flow of retained assumptions, constructive existence proofs, and representation costs inside coefficients.

This report contributes specific source-level diagnoses, exact countermodels, constructive repairs, and reproducible artifacts. It does not relabel old findings as new, turn historical acceptance counts into current results, or present independent Python calculations as execution of the Wolfram Language package.

The companion archive includes this article in source and rendered form, independent reference implementations, candidate Wolfram Language helpers, a snapshot-based characterization runner, raw test records, and a novelty ledger. No repository files were modified during this review.

Contents

1	Executive assessment	3
2	Scope, source grounding, and nonduplication	4
2.1	The reviewed object	4
2.2	Evidence vocabulary	5
2.3	How the prior reviews were used	5
3	Architecture and comparison with Wolfram Language	5
3.1	What the package actually contributes	5
3.2	A fair native comparison	6
3.3	Compare equal mathematical requests	7
3.4	Native delegation is not interpreter substitution	7
4	Mathics compatibility: present strengths and evidence limits	7
4.1	The isolation design is appropriate	7
4.2	Compatibility is multidimensional	8
4.3	Do not turn historical receipts into current acceptance	8
5	M01: ProductLog has more than one compatibility failure mode	9
5.1	The mathematical and interchange contracts	9
5.2	An exact countermodel and a numerical non-conclusion	9
5.3	What the current workaround covers	10
5.4	A nonprincipal exact-core witness	10
5.5	Repair and acceptance obligations	10
6	M02: retained assumptions stop before polynomial-domain inference	11
6.1	The missing parameter	11
6.2	A complete mathematical oracle	11
6.3	The public comparison and the appropriate repair	12
7	M03: replace fixed radius guesses by a constructive witness	12
7.1	Why seven radii cannot establish eventual completeness	13
7.2	An exact polynomial radius certificate	13
7.3	Relations, conjunctions, and ordered domains	14
7.4	A crucial integration distinction: eventual is not fixed-interval	14
7.5	Implementation and test boundary	14
8	D-P06: sparse outer jets can hide dense inner polynomials	15
8.1	A separate allocation dimension	15
8.2	Why the public budgets do not control it	15
8.3	A sparse alternative and its real cost model	16

8.4	Acceptance and architectural placement	16
9	D-V01 and T01: what a portable test receipt can prove	17
9.1	The current runner has important safeguards	17
9.2	Endpoint observations do not establish temporal immutability	17
9.3	Reuse an existing solution rather than invent another framework	17
9.4	T01: positive is not the same as finite positive	18
10	Two bounded extension and semantic items	18
10.1	L01: the Limit adapter silently chooses a side	18
10.2	G01: terminating hypergeometric polynomials need no convergence gate	19
11	API and development recommendations tied to the new findings	19
11.1	Make the failing contract visible, not merely the function name	19
11.2	A dependency-ordered implementation plan	19
11.3	Testing the boundaries rather than inflating a function catalog	20
12	Reproducibility and supplied artifacts	20
12.1	What actually ran	20
12.2	Artifact map	21
12.3	Commands	21
13	Conclusions	22
A	Novelty crosswalk and evidence boundaries	23
B	Public characterization catalog	23
C	Source inspection inventory	24

1 Executive assessment

The most useful interpretation of AsymptoticAnalysis is an explicit algebra of asymptotic information, with several specialized inverse engines and an interoperability layer. A result can retain a finite expression, a scale, a remainder, branch information, a source model, and enough provenance to support later operations. The ordinary power-log representation is complemented by logarithmic, Fourier, Gamma/Barnes, flat-sector, and exact exponential-core constructions. This is a materially different objective from returning only a plausible leading expression. The public definitions and inspected specialized modules support this architectural reading [6, 7, 8, 9].

The repository is also changing rapidly. Its review directory contains three waves of nine reports. The maintained registers explicitly distinguish current fixes from historical observations and unresolved proposals. The latest source already contains fixes and compatibility work absent from the third-wave snapshot. Consequently, a useful fourth inspection must not start by copying earlier complaints about dense SeriesData, captured assumptions, or missing CI. This report uses the implementation register and wave-three crosswalk as an exclusion map, then follows newer code paths [2, 3, 4].

The main new technical result is a more precise account of the Mathics boundary. Inspection of Mathics 10.0.1 shows that its two-argument ProductLog problem is not adequately explained as a numerical mpmath argument swap. The exact-expression SymPy conversion also preserves the wrong argument order, while the inherited mpmath arity gate admits only one argument. These are different failure modes. The package’s principal-branch construction-site workaround helps selected internal expressions but does not provide a general symbolic or numerical bridge [18, 28, 29].

Two additional compatibility gaps have small, exact repairs. The polynomial real-domain adapter requires numerically recognized real coefficients, even when a symbolic coefficient is proved real by the request’s retained assumptions. The eventual-condition adapter tries only seven predetermined radii before falling back to a general quantifier mechanism. It therefore has no constructive route for a perfectly valid but sufficiently narrow neighborhood. A parameter-aware coefficient proof and a leading-coefficient radius certificate address these particular gaps without weakening mathematical admission [17, 11, 12].

The performance contribution is deliberately a delta to existing budget findings: *sparse logarithmic coefficient polynomials can still be made dense inside the coefficient layer*. The Mathics CoefficientRules adapter constructs a full CoefficientList, and the common Fourier implementation also uses dense coefficient lists. Bounding source support or Fourier frequencies does not bound this work. The supplied sparse reference implementation operates on a two-term polynomial of degree 10^8 without allocating $10^8 + 1$ coefficient positions. That is an independently executed representation test, not a native package benchmark [19, 7].

Finally, the portable runner’s before/after source comparison is weaker than its natural-language interpretation: it detects unequal observed endpoint contents, not every change during execution. Its test script is frozen, but its package source is read from the original tree. An independently executed A–B–A countermodel establishes the distinction. The repository’s own definition-comparison workflow already uses snapshots; the recommendation is to reuse that design, not invent another generic validation framework [26, 13].

ID	Contribution	Priority	Evidence here
M01	ProductLog exact conversion, numeric arity, and construction-site interception are distinct compatibility problems.	High	Source plus executed independent bridge countermodels; package probes unrun.
M02	Polynomial-domain inference does not consume retained parameter assumptions.	Medium	Source plus exact inverse and monotonicity oracles.

ID	Contribution	Priority	Evidence here
M03	Seven fixed radii leave a scale-dependent proof gap; a constructive polynomial radius removes it.	Medium	Source, proof, and exact reference tests.
D-P06	Sparse logarithmic coefficient degree can dominate allocation despite tiny outer support.	Medium	Sharpened prior resource finding; source and sparse reference tests.
D-V01	Endpoint equality is not an immutable-source execution record.	Medium	Sharpened validation finding; source and executed A–B–A countermodel.
L01	Automatic direction is mapped to a one-sided Limit operation.	Low / latent	Source and mathematical distinction; no public wrong expansion shown.
G01	Terminating hypergeometric polynomials are excluded by the adapter’s nonterminating convergence gate.	Extension	Source and exact finite-polynomial oracle; public bypass may exist.
T01	Nonfinite timeout values pass the portable runner’s positivity-only test.	Low	Source and tested candidate validation helper.

No new wrong coefficient or false remainder has been reproduced by running this package in the present environment. That limitation does not erase the source findings, but it changes their acceptance status. The article separates a demonstrated mathematical mechanism from the still-required public-entry reproduction.

2 Scope, source grounding, and nonduplication

2.1 The reviewed object

The review pins commit 513917b. Its full identifier appears on the title page and in the artifact metadata. The commit timestamp is September 10, 2026, 02:16 UTC, which is September 9 in the user’s Pacific time zone. All repository references in this article use that commit rather than a moving `main` link. Upstream Mathics implementation references use the 10.0.1 tag, matching the repository’s stated target [1, 13].

The package is now named `AsymptoticAnalysis`. This does not imply that the public inverse constructor was renamed: `AsymptoticInverse` remains a public operation. There are a modular entry point under `src/Kernel` and a generated standalone entry point at the repository root. The Mathics documentation describes a 54-module candidate in its latest definition-comparison discussion. The inspection concentrated on the public core, inverse branch and expression routing, Fourier coefficient algebra, Gamma and exponential-core consumers, compatibility adapters, and validation machinery [6, 13].

Repository content was retrieved through the GitHub connector. A complete local checkout was not obtained because container network access failed. There was no local Mathics installation or Wolfram kernel, and the available remote Wolfram service failed to connect. The analysis therefore does not claim a locally rebuilt standalone package, a repository-wide executed test suite, a full source-tree census, or an exhaustive read of every historical review PDF.

2.2 Evidence vocabulary

S: source observation identifies an inspected definition, control-flow path, or stated contract. **D: derivation** is a mathematical argument supplied here. **E: executed independent evidence** means the bundled Python/SymPy/mpmath reference code or synthetic tooling tests actually ran. **U: unrun integration probe** is a Wolfram Language characterization or candidate helper supplied for later execution. Historical repository records are attributed as such and are not marked E.

These categories answer different questions. A source path can show that assumptions are not passed to a helper without proving exactly which failure message a particular interpreter prints. A reference calculation can prove an argument-order counterexample without establishing the package's complete evaluation sequence. A process can complete without satisfying a mathematical postcondition. The supplied kernel runner deliberately reports `Completed`, not `Success`, for a zero exit code.

2.3 How the prior reviews were used

The review index states that 167 entries are attributed across 27 packages, before consolidating overlaps. The main implementation register covers waves one and two and links all their local IDs; the wave-three intake provides the corresponding 44-entry crosswalk and unnumbered proposals. These registers, relevant source ranges, and the original review-six ledger were read to locate overlaps. The novelty decision is thus grounded in the maintained crosswalk and selected original material, not a claim that all 27 articles were independently re-reviewed end to end [2, 3, 4, 5].

Several tempting findings are intentionally omitted from the new-defect list. The existing register already covers refinement precision demands, semantic exponent equality, native-tail admission, eager native normalization, option-key/default routing, result-schema validation, target-chart metadata, logarithmic normalization branches, and certificate accuracy. Their appearance in current source is not sufficient reason to publish them again. Likewise, the broad goals of complete native coverage, full Mathics compatibility, and support for the vendored asymptotic corpus are already project requirements [3, 4].

The two explicitly prefixed delta items retain their ancestry. D-P06 sharpens the earlier resource/budget discussion with a different allocation dimension and concrete coefficient-layer consumers. D-V01 sharpens the earlier validation discussion with an exact temporal countermodel and a mismatch between two current validation approaches. M01 is also adjacent to the previously documented unchecked numeric accessor, but its new content is the interpreter-specific symbolic/numeric bridge analysis, not a repeat of the general recommendation to validate numerical domains.

3 Architecture and comparison with Wolfram Language

3.1 What the package actually contributes

An ordinary power-log object represents a finite expression of the form

$$F(u) = \sum_{\beta < h} u^\beta P_\beta(\log u) + O\left(u^\rho (1 + |\log u|)^k\right), \quad u \downarrow 0, \quad (1)$$

with exact real exponents and polynomial logarithmic coefficients. The local coordinate can encode a finite endpoint and side, or a signed infinity. The core exports terms, remainder metadata, an ordinary expression, and model/provenance information. Specialized engines change the scale rather than pretending that every expression lies on a single rational-power lattice [6, 7, 8, 9].

This information is useful only when preserved under operations. Addition, multiplication, logarithms, powers, composition, observables, truncation, refinement, and qualified differentiation are exposed as explicit operations; ordinary arithmetic is also routed through the object interface. Differentiation is not justified from a magnitude bound alone: the public contract asks for matching derivative information. Inverse residuals distinguish a finite-model identity from an interval certificate for an actual numerical root. These distinctions are strengths worth retaining during compatibility work [6, 3].

The Fourier layer adds a second algebra inside each source-power block:

$$P_\beta(L) = \sum_{\omega \in \Omega_\beta} Q_{\beta,\omega}(L) e^{i\omega L}, \quad L = \log u. \quad (2)$$

It explicitly tracks frequencies and their real-conjugacy relation. The Gamma/Barnes inverse layer instead organizes complete inverse-logarithmic polynomials around an exact Lambert core. The exponential-core layer can retain an exact inverse while expanding perturbative sectors around it. A correct implementation must preserve both the outer grading and the inner coefficient algebra; the latter becomes central to D-P06 [7, 8, 9].

3.2 A fair native comparison

The relevant comparison is not “the package versus Taylor series.” Wolfram Language already provides a substantial family of asymptotic tools. The following table describes documented interfaces, not measured superiority or failure on the supplied probes.

Native facility	Documented capability	Meaning for this package
Series	Taylor expansions and selected negative-power, fractional-power, and logarithmic expansions; finite/infinite centers and successive multivariate expansions [30].	An essential baseline, not a toy competitor. Package comparisons must match the requested cutoff and the meaning of the remainder.
SeriesData	A rational exponent lattice with coefficient storage and native arithmetic, composition, differentiation, and integration [31].	Efficient interoperability on suitable inputs; not the package’s full generalized-exponent or specialized-scale representation.
InverseSeries	Reversion of an appropriate SeriesData object, including documented nontrivial leading-power examples [32].	Must be included in ordinary/ramified inverse comparisons. It should not be described as limited to the most elementary regular Taylor case.
Asymptotic	Asymptotic approximations for expressions and supported analytic objects, with automatically chosen scales and native options [33].	Much broader native problem coverage. Delegation preserves that output but is not a proof of the package’s analytic remainder contract.
AsymptoticSolve	Asymptotic equation solving, including systems, parameter expansions, and real or complex settings [34].	A genuine inverse/implicit-equation competitor. This review makes no claim that it fails the generalized inverse examples below.
DiscreteAsymptotic	Integer-at-infinity asymptotics, including supported sums, products, coefficient sequences, and recurrence-solution expressions [35].	The package’s missing direct backend is already a tracked coverage obligation, not a newly discovered issue.

The package’s strongest differentiator is therefore *the explicit organization and reuse of asymptotic information on admitted scales*, not the absence of asymptotic algorithms in the host language. Conversely, native functionality is broader in problem classes and ambient mathematical settings. The package’s real-branch policy and specialized constructors should be compared against that breadth honestly, without presenting a representation restriction as a universal mathematical limitation of Wolfram Language.

3.3 Compare equal mathematical requests

The ordinary package cutoff retains exponents strictly below h . Native `Series` uses its own order convention. For the mathematical expansion of e^x , a package cutoff of three requests

$$1 + x + \frac{1}{2}x^2 + O(x^3),$$

whereas native integer order three includes the cubic term. The appropriate ordinary comparison is therefore with native order two, subject to the chosen approach and remainder contract. These are specification examples, not printed outputs obtained in this review [6, 30].

A second comparison dimension is the exponent set. Let $\alpha > 1$ be fixed and real, and solve $y = x + x^\alpha$ on the positive germ. Writing $x = yv$ and $z = y^{\alpha-1}$ gives

$$1 = v + zv^\alpha.$$

Coefficient matching at $v = 1$ yields

$$x = y - y^\alpha + \alpha y^{2\alpha-1} - \frac{\alpha(3\alpha-1)}{2} y^{3\alpha-2} + \dots \quad (3)$$

For $\alpha = \sqrt{2}$, these exponents are not a single fixed rational `SeriesData` lattice in y . This explains the utility of a generalized representation. It does *not* establish that native `Asymptotic` or `AsymptoticSolve` cannot obtain such an approximation. The mathematical derivation and the representation comparison are separate claims [31, 34].

Similarly, retaining an exact core can be useful even when an elementary or Lambert expression is already known. It prevents an outer perturbation expansion from repeatedly re-expanding that core. In exchange, later numerical evaluation inherits the runtime's ability to interpret that exact expression correctly. The expression *looks* portable while its evaluator may not be; M01 concerns precisely this boundary.

3.4 Native delegation is not interpreter substitution

The current public API includes explicit `"Series"`, `"Asymptotic"`, and `"Package"` backend choices and automatic routing. Native results retain a distinct kind/scale and do not acquire an analytic package remainder merely because a native computation returned an expression. Automatic routing already exists; claims from older reviews that all delegation must be explicit are obsolete for this snapshot [6, 3].

On `Mathics`, selecting a native backend selects `Mathics`'s implementation, not the official `Wolfram` implementation. Creating an inert `System'` name permits parsing and symbol agreement but supplies no solver. Thus even a complete dispatcher cannot, by itself, satisfy the goal of subsuming every successful official-kernel input on another interpreter. The actual computational or theorem-producing capability has to be implemented, delegated to a genuinely available service, or truthfully reported as unavailable [13, 15].

4 Mathics compatibility: present strengths and evidence limits

4.1 The isolation design is appropriate

The package does not indiscriminately replace global `System'` definitions. Its compatibility modules are loaded conditionally and use a package-owned context; the ordinary `Wolfram` path keeps its own bindings. The `Mathics` bootstrap creates selected inert public-input names so that option and function symbols agree across caller and package contexts. This is a sensible compromise between parser compatibility and evaluator substitution, provided that symbol existence is never used as proof of implementation [15, 13].

The adapters are more substantive than a collection of aliases. They address explicit `Module`-targeted returns, held extraction for named functions, association access and updates, empty-list mapping, exact realness/sign consequences, finite defining Taylor expansions, Stirling modeling, formatting installation, and interpreter-specific deadlines. Several comments carefully distinguish a conservative unknown answer from

a proof. In particular, the current assumption layer checks the realness of operands before permitting native simplification of ordered predicates; this avoids manufacturing a real inequality by canceling a common complex offset [15, 23, 24, 16, 20, 21, 22].

These are good boundaries to preserve. Replacing them with aggressive PowerExpand, numerical sampling, or global built-in patches would trade visible coverage gaps for difficult-to-audit incorrect results. The repairs proposed below add narrowly scoped mathematical facts rather than weaken the existing admission policy.

4.2 Compatibility is multidimensional

Layer	Evidence in the pinned repository	Remaining qualification
Load and symbol binding	Conditional bootstrap, private adapters, generated standalone support, reload handling.	Successful loading is not mathematical compatibility.
Language primitives	Bounded Return/Module, association, Extract, Map, and formatting workarounds.	Only the admitted forms should be treated as replaced semantics.
Algebra and assumptions	Exact consequence rules and guarded native simplification.	Symbolic parameter proofs and construction-site assumption flow still have gaps.
Forward special functions	Restricted defining series and finite positive-argument Stirling models.	Arity, parameter, argument-chart, order, and convergence restrictions matter.
Inverse branches	Polynomial real-domain and affine-interval facts supplement ordinary validators.	These are not general quantifier elimination; M02 and M03 expose particular missed cases.
Numerical interpretation	Selected exact quadratic and principal Lambert smoke checks are documented.	Symbolic availability does not establish nonprincipal Lambert numerical behavior.
Remainders and certificates	Selected exact examples and conservative contracts are documented.	A finite model, numerical residual, and original-function enclosure remain different evidence.
Display	Package-specific formatting; Normal and InputForm provide alternatives.	Front-end box support and evaluator correctness must be tested separately.

The table summarizes the repository's stated and inspected scope, not tests executed in this session [13, 25, 17].

4.3 Do not turn historical receipts into current acceptance

The Mathics compatibility document reports an earlier official-Wolfram preservation run with 1,452 passes and the same 12 pre-existing failures across 79 suites. It also describes later modular and standalone definition comparisons, with 2,051 and 2,050 package symbols respectively, against a newer control. The document explicitly declines to relabel the older full-suite run as a full run of later merged source. These are appropriately scoped historical statements; they are not a current Mathics acceptance result [13].

The current Mathics workflow exists and should receive credit. It separates modular and standalone entry points, runs five suite groups in each, installs the stated Python 3.11 environment, performs generation checks, and uses fresh-kernel portable regressions with process deadlines. The compatibility document still

says that consolidated final-source acceptance is pending. No current workflow execution was independently inspected or rerun here [27, 13].

The environment used for the independent artifacts here is different: Python 3.13.5, SymPy 1.14.0, and mpmath 1.3.0, without Mathics. That is suitable for exact reference algorithms and bridge countermodels, but it must not be described as testing the repository’s Python 3.11/Mathics 10.0.1 target stack.

5 M01: ProductLog has more than one compatibility failure mode

Evidence and status. S/D/E/U. The source mapping and arity facts are directly inspected. The numerical examples in this section were executed independently in SymPy/mpmath. No output is attributed to a running Mathics or AsymptoticAnalysis instance.

5.1 The mathematical and interchange contracts

Write $W_k(z)$ for branch k of the Lambert function, so that

$$W_k(z)e^{W_k(z)} = z. \quad (4)$$

Wolfram’s two-argument spelling is `ProductLog[k,z]`. The corresponding SymPy constructor uses the argument first, `LambertW(z,k)`. A bridge must therefore preserve branch identity while exchanging positions in both conversion directions. A principal one-argument form can be normalized separately. The bundled reference conversion implements that contract, rather than calling the two constructors with the same positional list [28, 29].

The current package comments attribute the Mathics two-argument problem to forwarding Wolfram-order arguments to mpmath. Following the upstream implementation gives a sharper diagnosis. The Mathics `ProductLog` class inherits its exact conversion behavior from `SympyFunction`; the default `prepare_sympy` does not reorder the operands. The exact evaluation route calls that conversion before converting a result back. Meanwhile, the inherited `MPMathFunction` default declares `nargs = \{1\}`; the `ProductLog` class does not override it to admit two numerical arguments [18, 28, 29].

Thus there are at least three distinct contracts to test: exact conversion, conversion back after symbolic transformations, and inexact numerical evaluation. A claim about one is not evidence for the other two.

5.2 An exact countermodel and a numerical non-conclusion

The principal identity $W_0(e) = 1$ gives an especially small countermodel. An unreordered exact bridge sends `ProductLog[0,E]` to `LambertW(0,E)`, not to `LambertW(E,0)`. The independent SymPy evaluation recorded in the archive is

$$\text{LambertW}(e, 0) = 1, \quad \text{LambertW}(0, e) = -\infty.$$

The second expression is the bridge countermodel, not a legitimate interpretation of the requested Wolfram expression.

It would nevertheless be wrong to infer that every inexact two-argument Mathics call necessarily takes the same bad numerical route. The inspected mpmath arity check can reject the two-argument call and leave it unevaluated. This matters operationally: “wrong value,” “wrong exact simplification,” and “inert expression” need different regression expectations and fixes. The earlier package comment does not capture that distinction [29, 28].

A no-op symbolic round trip can also be misleading. If both directions preserve the same wrong ordering, an uninterpreted pair may come back looking unchanged. That does not prove that simplification or numerical specialization used the intended function in the intermediate representation. Acceptance tests must examine the *intermediate semantics*, not only structural equality before and after a round trip.

5.3 What the current workaround covers

The package's late Mathics helper normalizes branch zero to the one-argument System form, while nonzero branches are passed through. A definition rewrite applies the helper at four named construction sites: automatic core inversion, exponential-core exact inversion, Lambert construction, and special thresholds. The code is intentionally isolated and avoids changing global System definitions [18].

This improves those internal principal constructions. It is not a parser-wide interception of every `ProductLog[0, z]` supplied by a caller, nor a numerical implementation of W_{-1} . A held public constructor eventually evaluates source expressions in its private entry path. The following supplied characterization compares mathematically equivalent source constants:

```
AsymptoticExpansion[x + ProductLog[0, E], {x, 0, 3},
  "Backend" -> "Package"]
AsymptoticExpansion[x + ProductLog[E], {x, 0, 3},
  "Backend" -> "Package"]
```

The mathematically correct finite expression is $1 + x$. Whether the first call is rejected after native pre-evaluation, remains unresolved, or is normalized elsewhere is an integration observation still to be made. The probe is not accompanied by fabricated package output.

5.4 A nonprincipal exact-core witness

Consider

$$f(x) = \frac{e^x}{x}, \quad x > 1.$$

Its derivative is $e^x(x - 1)/x^2 > 0$, and the increasing inverse is

$$g(y) = -W_{-1}(-1/y), \quad y > e. \quad (5)$$

At $y = e^2/2$, the correct value is exactly 2. The inspected exponential-core constructor selects branch -1 when its power parameter b is negative; e^x/x is the case $a = c = p = 1, b = -1$. With zero perturbation, retaining this exact core is mathematically appropriate [9].

The generalized-series numeric accessor, however, performs direct substitution into the stored expression outside the Native kind. It has no interpreter-specific nonprincipal Lambert guard [6]. This creates a concrete boundary to characterize:

```
s = AsymptoticExponentialCoreInverse[
  Exp[x]/x, 0, {x, Infinity}, {y, 0}];
{Normal[s], s[N[E^2/2, 30]]}
```

The source and mathematics establish what must be preserved. They do not establish the exact current Mathics printout. For illustration only, independently evaluating the *swapped SymPy expression*

$$-\text{LambertW}(-1, -2/e^2) \approx 0.3181315052 - 1.3372357014i$$

gives neither the correct branch nor the correct real value. This is recorded as a SymPy countermodel, explicitly *not* as observed package output.

5.5 Repair and acceptance obligations

The durable fix belongs at the upstream interchange boundary: convert `ProductLog[k, z]` to `LambertW(z, k)`, invert that transformation when returning, admit the numerical arities actually supported, and validate the branch argument rather than allow it to be confused with the evaluation point. Principal and nonprincipal tests must cover exact input, arbitrary-precision input, symbolic transformations, and numerical specialization after storage. Fixing only one conversion direction is insufficient.

For the package, an interim repair should remain package-owned. A narrowly admitted literal principal-branch form can be normalized while the source is held, before ordinary evaluation. Nonprincipal symbolic expressions can remain available where their manipulation is proved safe, but a package numerical operation must either use a tested bridge or return a clear capability refusal. This is not a proposal to globally patch Mathics's System function. A general unchecked external operation on `Normal[s]` cannot be made safe solely by guarding `s[value]`; that limitation should remain explicit.

Acceptance requires the principal control above, the exact-core point 2, branch--1 fixed-point residuals at several negative arguments in $(-1/e, 0)$, and symbolic round trips that inspect branch positions. Near the branch point, numerical conditioning deserves a separate tolerance policy. A value with a small residual alone is not a proof that the desired branch was selected.

Relationship to earlier reviews. The old D05/R6-A10 item already notes that the numeric accessor is unchecked. M01 does not repeat that finding. It adds the exact SymPy conversion path, the inherited one-argument numeric gate, the limited construction-site interception, and a branch-specific exact-core witness. It also corrects the package's mpmath-only diagnosis.

6 M02: retained assumptions stop before polynomial-domain inference

Evidence and status. S/D/E/U. The missing assumption argument is visible in the adapter. Exact monotonicity and inverse-coefficient checks were run independently; the public callable comparison remains unrun.

6.1 The missing parameter

The Mathics polynomial-domain helper proves that a polynomial with numerically recognized exact real coefficients has domain all of $\mathbb{R}$. Its coefficient condition uses the core's `exactRealQ`, which includes `NumericQ`. It takes three arguments: body, source symbol, and Reals. The late rewrite substitutes this helper for `FunctionDomain` inside branch selection [17, 6].

The branch-selection routine itself has access to the request's retained assumptions. The helper does not receive them. A symbolic coefficient a therefore does not become admissible merely because the request states $a > 0$. This is not the old ambient-assumption leakage problem: assumptions can be properly retained at the object/request boundary and still be unavailable at a particular proof call. The specific defect is an overly restrictive proof interface [11, 3].

6.2 A complete mathematical oracle

Let $a > 0$ and

$$f_a(x) = ax + x^3. \quad (6)$$

Every coefficient is real, the polynomial is defined for all real x , and

$$f'_a(x) = a + 3x^2 > 0.$$

Together with its limits at $\pm\infty$, this gives a unique increasing real inverse. Around zero it has

$$g_a(y) = \sum_{n \geq 0} \frac{(-1)^n}{(2n+1)a^{3n+1}} \binom{3n}{n} y^{2n+1}. \quad (7)$$

For completeness, apply ordinary Lagrange inversion to $y = x(a + x^2)$. The coefficient of y^m is $m^{-1}[x^{m-1}](a + x^2)^{-m}$. Odd $m = 2n + 1$ gives

$$\frac{1}{2n+1} a^{-(2n+1)} a^{-n} \binom{-2n-1}{n} = \frac{(-1)^n}{(2n+1)a^{3n+1}} \binom{3n}{n};$$

even coefficients vanish. In particular,

$$g_a(y) = \frac{y}{a} - \frac{y^3}{a^4} + \frac{3y^5}{a^7} - \frac{12y^7}{a^{10}} + \frac{55y^9}{a^{13}} + \dots$$

The independent test substitutes this polynomial into $ag + g^3 - y$ and verifies cancellation through degree ten. The analytic inverse exists near zero because $f'_a(0) = a \neq 0$; the series is not being asserted uniformly as $a \downarrow 0$.

6.3 The public comparison and the appropriate repair

The supplied probe compares an inverse callable passed to forward expansion with the direct inverse constructor:

```
AsymptoticExpansion[InverseFunction[Function[t, a t + t^3]],
  {y, 0, 6}, Assumptions -> a > 0, "Backend" -> "Package"]
AsymptoticInverse[a x + x^3, {x, 0}, {y, 6},
  Assumptions -> a > 0]
```

Native pre-evaluation of `InverseFunction` can differ between runtimes, as the repository already documents. The test must therefore retain the original call and classify whether the specialized branch path was actually reached. The source finding does not assume that every syntactic form produces the same failure message [14, 12].

A suitable replacement hook takes the assumptions explicitly:

```
polynomialRealDomain[body, source, assumptions]
```

It should first establish polynomiality in the source variable, then prove each coefficient finite real under parameter-only assumptions. If any coefficient proof remains unknown, the result remains unknown. If the body is not polynomial, the existing general fallback remains available. No ambient default should silently strengthen the retained domain.

The companion `PolynomialRealDomain` prototype follows this design and uses the supplied sparse coefficient extractor. It returns `True` only on proof and `None` otherwise. It has not been run in either Wolfram or Mathics and is not installed by the archive. The Python oracle verifies the mathematical example, not that prototype's evaluator behavior.

The minimum acceptance set is $a > 0$, $a \in \mathbb{R}$ for domain-only inference, no assumptions, an explicitly nonreal coefficient, and a parameter that depends on the source. Domain inference and monotonicity must remain separate: proving $a \in \mathbb{R}$ makes (6) real everywhere but does not prove that it is globally increasing for every such a .

Relationship to earlier reviews. C05 already concerns retaining assumptions. M02 identifies a newer Mathics consumer that does not accept them at all and gives an exact, parameterized inverse oracle. Its repair is coverage-preserving proof plumbing, not a relaxation of the real-branch contract.

7 M03: replace fixed radius guesses by a constructive witness

Evidence and status. S/D/E/U. The seven-trial schedule is inspected. The radius theorem is proved below, with 400 exact polynomial cases and additional targeted checks in the independent test suite. No general Mathics quantifier fallback was executed here.

7.1 Why seven radii cannot establish eventual completeness

The Mathics replacement for `inverseFunctionEventually` simplifies under $u > 0$, then tries affine interval proofs at

$$1, \frac{1}{2}, \frac{1}{4}, \dots, \frac{1}{64}.$$

If no trial succeeds, it invokes the original eventual-condition mechanism. That original path asks for an existential positive radius and a universal condition throughout the resulting deleted neighborhood, using general symbolic quantifiers. Those facilities are not supplied by the bounded affine adapter [17, 12].

For the condition $u < 1/1000$, every listed radius extends beyond the valid interval. The correct affine interval answer at each of those radii is inconclusive, not true. Nevertheless, the condition holds for every sufficiently small positive u . The same issue occurs at arbitrarily small rational margins and, more importantly, for the parameterized condition $u < \eta$ under $\eta > 0$: no single fixed trial radius works for every admitted parameter value.

The following public source pair has the same local germ and different distant cutoffs:

```
AsymptoticExpansion[ConditionalExpression[x, x < 1/1000],
  {x, 0, 3}, "Backend" -> "Package"]
AsymptoticExpansion[ConditionalExpression[x, x < 1],
  {x, 0, 3}, "Backend" -> "Package"]
```

Both are mathematically the identity near zero from above. The bounded trial mechanism can prove the second condition at radius one but not the first at any of its seven radii. Whether some other runtime simplification discharges the first call is left to the supplied integration probe. The independent oracle models only the explicit seven trials and labels that scope in its docstring.

7.2 An exact polynomial radius certificate

The gap does not require a general quantifier-elimination implementation. A small leading-coefficient theorem suffices for polynomial predicates.

Proposition 7.1 (Constructive eventual sign). *Let $P \in \mathbb{R}[u]$ be nonzero, and write*

$$P(u) = u^m \left(c + \sum_{j>m} c_j u^{j-m} \right), \quad c \neq 0,$$

where m is the least retained exponent. Set

$$B = \sum_{j>m} |c_j|, \quad r = \min \left(1, \frac{|c|}{2(1+B)} \right). \quad (8)$$

Then $r > 0$, and $P(u)$ has the sign of c throughout $0 < u < r$.

Proof. For $0 < u < r \leq 1$ and each integer $j - m \geq 1$, one has $u^{j-m} \leq u$. Hence

$$\left| \sum_{j>m} c_j u^{j-m} \right| \leq uB < \frac{|c|}{2}$$

when $B > 0$, using $rB \leq |c|B/(2(1+B)) < |c|/2$. For $B = 0$ the tail vanishes. The bracket cannot change the sign of c , and $u^m > 0$ on the stated interval. $\square$

All quantities can remain exact. For $P(u) = 1/1000 - u$, the formula gives $r = 1/4000$, immediately certifying a positive neighborhood. It is intentionally conservative; it is a proof-producing bound, not an

attempt to compute the maximal interval. Very small leading coefficients affect the size of the exact rational radius, not the number of trial guesses.

For symbolic coefficients, the same formula is valid under retained assumptions that prove the relevant coefficients real and the leading coefficient nonzero with a determined sign. The returned radius may depend on parameters. This proves a pointwise family of eventual statements, not a uniform radius or uniform asymptotic constant across the parameter domain. If the leading coefficient might vanish, the helper must return unknown or perform an independently justified case split.

7.3 Relations, conjunctions, and ordered domains

Apply Proposition 7.1 to the difference of the two sides of a binary polynomial relation. Positive, negative, and identically zero differences determine the eventual truth of $<$, $\leq$, $>$, $\geq$, $=$, $\neq$. A conjunction of certified predicates is certified on the minimum of their positive radii. An ordered chain is treated through its adjacent comparisons; an all-distinct predicate requires all pairs, not merely adjacent pairs.

For an *ordered* relation, both original operands must first be proved real in the local domain. Proving only that their difference is real is not enough. For example, canceling a common imaginary offset does not turn an order comparison of complex values into an admitted real inequality. The repository already guards this issue in its assumption and affine-interval adapters; the proposed certificate must retain that guard [16, 17, 20].

The zero polynomial is handled separately and has no leading nonzero coefficient. Unknown symbolic cancellation is likewise not permission to delete a coefficient. In a bounded implementation, inability to establish the leading term produces an unresolved proof request, not a guessed sign.

7.4 A crucial integration distinction: eventual is not fixed-interval

There are two different questions in the branch code:

$$\exists r > 0 \forall u \in (0, r) : C(u), \quad \text{and} \quad \forall u \in (0, r_0) : C(u)$$

for a *previously chosen* r_0 . The new radius certificate answers the first. It may answer the second only if the returned witness is at least r_0 , or after the surrounding algorithm explicitly shrinks its recorded domain and propagates that change.

For instance, $1 - 1000u > 0$ is eventually true but not true throughout $(0, 1)$. It would be unsound to replace the fixed-interval `inverseBranchEventualQ` by a Boolean eventual test and keep the old radius metadata. The independent suite includes this counterexample specifically to prevent an attractive but incorrect patch.

The simplest integration is therefore to use the new result first in `inverseFunctionEventually`, whose contract is genuinely existential. A later integration into branch-validation intervals should return a certificate containing the actual radius and update all dependent interval statements. This scope distinction is more important than making the radius numerically sharp.

7.5 Implementation and test boundary

The Python implementation supports sparse polynomials with exact rational coefficients and returns a structured certificate with the sign, radius, valuation, leading coefficient, and coefficient majorant. Its verifier checks the stated sufficient inequality. The Wolfram Language companion is intentionally narrower: it prototypes binary affine comparisons with explicit assumptions. It is not a complete implementation of arbitrary polynomial or Boolean predicates.

The executed evidence consists of exact narrow-margin examples, margins down to 2^{-1000} in the targeted affine tests, 400 deterministic random polynomial cases, three exact interior evaluations per random case, and majorant verification. Sampling alone would not prove the proposition; the proof above supplies that justification, while the exact evaluations help detect implementation mistakes. The code keeps a crossing predicate's fixed-interval answer distinct from a universally false predicate.

Relationship to earlier reviews. The earlier registers discuss conservative proof and branch admission generally, but not this new seven-radius adapter and its scale-dependent coverage gap. The constructive certificate and the prohibition against silently replacing a fixed-interval theorem are the substantive additions.

8 D-P06: sparse outer jets can hide dense inner polynomials

Evidence and status. S/D/E/U. The dense calls are inspected. Storage-position counts are exact consequences of the representations. The sparse reference algorithms were executed; no native timing, peak-memory measurement, or large-degree package stress run is claimed.

8.1 A separate allocation dimension

Let

$$Q(L) = 1 + L^D.$$

This polynomial has two nonzero coefficients, but a degree-indexed coefficient list has $D + 1$ positions. The distinction remains even if an implementation shares its zero objects: the list still needs $D + 1$ indexed positions. The degree D itself takes only $O(\log D)$ bits to encode.

The `Mathics CoefficientRules` adapter currently obtains its result by expanding the polynomial, constructing `CoefficientList`, pairing every position with its degree, and discarding zero entries. The final answer is sparse, but the intermediate representation is not. For this example, producing two rules passes through a degree-sized array [19].

The same dimension exists outside the `Mathics`-only layer. The common Fourier routine `fourierPoly` canonicalizes all entries of a `CoefficientList` and reconstructs the polynomial using powers indexed by the degree range. `fourierEnvelope` also enumerates a coefficient list. Thus the issue is not fixed by making the `Mathics` primitive sparse while leaving the Fourier consumer unchanged [7].

8.2 Why the public budgets do not control it

Equation (2) has at least three distinct sizes: the number of source weights β , the number of frequencies ω , and the support/degree of each $Q_{\beta,\omega}$. A source with few weights and two frequencies can still have an enormous logarithmic degree. A candidate family is

$$f_D(x) = x + x^2(\log x)^D \sin(\log x), \quad x \downarrow 0, \quad (9)$$

where D is a fixed nonnegative integer. For every fixed D , $x(\log x)^D \rightarrow 0$, so the correction is asymptotically smaller than the leading x . Differentiating the correction shows a factor x times a finite logarithmic polynomial/trigonometric expression, so the derivative tends to one. An increasing local inverse exists sufficiently near zero.

This family therefore tests a legitimate input dimension rather than merely a malformed expression. Parsing its trigonometric coefficient produces two frequency modes with high-degree polynomial amplitudes. A small `MaxFrequencies` does not bound D , and a bound on retained source blocks does not either. The relevant package entry for a bounded characterization is

```
AsymptoticFourierInverse[
  x + x^2 Log[x]^D Sin[Log[x]], {x, 0}, {y, 3},
  "MaxTerms" -> 20000, "MaxFrequencies" -> 256]
```

No claim is made that this call was executed or that it reaches the same final failure on every runtime. A regression should start at modest degrees and measure the parsing/canonicalization phase separately. Running an enormous D against the current dense implementation is not necessary to establish the allocation mechanism.

There is also a narrower consumer optimization. `gammaInverseBound` calls `CoefficientRules` to obtain the smallest nonzero inverse-logarithmic exponent. It needs a valuation, not a full degree-indexed

list. That call should be implemented as a direct sparse valuation query. This does not imply that the current Gamma constructor cheaply reaches degree 10^8 ; its preceding coefficient-generation work has its own cost. The call identifies unnecessary representation work, not a demonstrated end-to-end Gamma benchmark [8].

8.3 A sparse alternative and its real cost model

Store a polynomial as a sorted collection

$$\{(d_j, c_j) : j = 1, \dots, s\}, \quad 0 \leq d_1 < \dots < d_s, \quad c_j \neq 0.$$

Coefficient rules and valuation then read this support directly. Differentiation maps (d, c) to $(d - 1, dc)$ for $d > 0$. An ℓ^1 envelope is the expression

$$\sum_j |c_j| M^{d_j},$$

which does not require inserting interior zeros. Addition merges equal degrees; multiplication convolves pairs with an explicit candidate-pair budget. Canonicalization of coefficients and equality grouping still need their own proof/cost policies.

For fixed coefficient bit-size, storing the support takes $O(s \log D)$ degree bits rather than $D + 1$ positions. This is a representation statement, not a universal runtime bound. Expansion of a factored expression can increase s dramatically, and coefficient simplification can dominate. A sparse extractor should therefore be paired with an input-expansion budget or a representation-aware traversal; merely replacing one list constructor does not solve all expression swell.

The supplied Python helper avoids a subtle trap in an independent oracle: it does not construct a SymPy Poly for the huge-degree case. Instead, after expansion, it reads monomial degree/coefficient pairs and rejects residual dependence on the selected variable. The separate exact-rational SparsePolynomial class has immutable sorted support, validation, cancellation, differentiation, multiplication, and valuation operations. The Wolfram Language companion provides the corresponding bounded extraction design but remains unrun.

Largest degree D	Dense positions $D + 1$	Nonzero support of $1 + L^D$
100	101	2
10 000	10 001	2
1 000 000	1 000 001	2
100 000 000	100 000 001	2

Table 1: Exact representation counts, not measured native allocations or timing. The sparse reference test constructs the last case without a degree-indexed array.

8.4 Acceptance and architectural placement

A patch should first replace the new Mathics primitive and the specific Fourier consumers, then verify coefficients, frequency conjugacy, zero-polynomial behavior, and remainder-envelope equivalence at ordinary degrees. A matched performance comparison should hold support and coefficient values fixed while increasing D , record failed cases, and separate extraction, canonicalization, convolution, and display. Returning the same two rules after a large allocation is not a successful sparse-performance fix.

An explicit logarithmic-degree limit is a possible temporary availability guard. It should return a resource failure, not silently remove a high-degree amplitude or count it as a negligible outer term. A high logarithmic degree changes the coefficient polynomial within a retained block; it is not the same as an exponent beyond the source-power cutoff.

Relationship to earlier reviews. C01's optional dense SeriesData export and W3-04's eager Normal problem are already recorded. This item concerns different internal representations and survives even when

those exports are disabled. It is a concrete refinement of P06/R6-A06 and P08, not a new claim that the repository has never considered budgets or sparse algebra.

9 D-V01 and T01: what a portable test receipt can prove

Evidence and status. S/D/E/U. The portable runner was read, but not launched with a native kernel. A source-mutation countermodel and the supplied patch/archive helper tests were executed independently.

9.1 The current runner has important safeguards

The portable Python runner uses fresh processes, validates declared case IDs, rejects an empty selection, and distinguishes explicit success/failure records from crashes, timeouts, launch errors, and protocol errors. It retains complete diagnostics. Its timeout handling terminates the owned process group or Windows process tree, rather than indiscriminately killing all kernel processes. It also freezes the Wolfram Language test script to avoid a streamed input file changing underneath an interpreter [26].

These are strong improvements over the legacy aggregate-runner concerns in older reviews. The problem below is narrower: it concerns what the source-integrity field means, not whether every test result can be trusted or whether the runner has no safeguards.

9.2 Endpoint observations do not establish temporal immutability

The runner computes fingerprints before execution and when it writes reports. It passes the original resolved package path to the test through an environment variable. Its frozen test script is therefore not paired with a frozen package tree. The field `SourcesUnchangedDuringRun` is calculated by comparing observed before/after contents [26].

Consider an ordinary file with initial contents A. Between two observations it is changed to B, the interpreter reads B, and the file is restored to A. The endpoint contents compare equal, yet the interpreter did not consume A. This requires no hash collision and no exotic filesystem semantics. It is simply a different temporal proposition:

$$\text{contents at } t_0 = \text{contents at } t_1 \quad \not\Rightarrow \quad \text{contents were constant on } [t_0, t_1].$$

The independent countermodel performs exactly these writes and reads, then records two Boolean facts: endpoint equality is true, and the read bytes differ from the initial bytes. It does not allege that such a change occurred in a repository acceptance run.

The distinction also matters without intentional interference. An editor, generator, checkout, or development script can change and restore files while a streamed load is in progress. A record should not promise a stronger invariant than the acquisition method supplies.

9.3 Reuse an existing solution rather than invent another framework

The repository’s Mathics definition-comparison documentation already describes copying package inputs and capture scripts to temporary snapshots, checking their identities, and rejecting changes. The portable runner should reuse that acquisition boundary. At minimum it should distinguish “original source contents matched at observations” from “these frozen inputs were executed” [13].

The companion characterization runner reads a resolved Git commit with `git archive`, extracts only ordinary files into a private temporary directory, copies the probe there, and loads the archived entry point. It supports both modular and standalone forms. This produces a coherent committed source snapshot and avoids pretending to test uncommitted working-tree edits. It is not a sandbox against hostile code running under the same operating-system account, and the report does not describe the temporary files as cryptographically immutable.

For production validation of working-tree changes, a coherent snapshot can be taken after generation, verified against the requested candidate, and then used exclusively by the kernel. The receipt should identify

that snapshot and its selected tests. Native definition equality, a selected runtime regression, and a full-suite run should remain distinct record types. The current compatibility document already makes that distinction well.

Relationship to earlier reviews. The older V01/V02 discussion already demands stronger test evidence and runner checks. D-V01 adds the particular A–B–A countermodel, the asymmetry between frozen script and live package inputs, and a concrete reuse opportunity in the repository’s newer comparison runner. It is not a generic request to add CI.

9.4 T01: positive is not the same as finite positive

The portable command-line parser converts its timeout argument to a Python float and rejects values satisfying `timeout <= 0`. That test accepts nan because ordered comparisons with NaN are false, and it also accepts positive infinity. Neither value is a meaningful finite hard deadline [26].

The input check should require

```
if not math.isfinite(args.timeout) or args.timeout <= 0:
    parser.error("--timeout must be positive and finite")
```

The archive supplies `patch_timeout.py`. By default it prints a unified diff; it writes only with an explicit `-apply`. It checks exact source fragments and refuses unexpected versions instead of performing an unbounded text substitution. Synthetic tests verify the intended change, idempotence, and refusal of changed/duplicate source fragments. The patch was not applied to the repository during this review.

This is a small input-validation defect, not evidence that normal positive timeouts fail. The snapshot-based review runner already uses the finite-positive check. Its subprocess layer remains an unrun integration component; the helper tests do not substitute for launching the target interpreters.

10 Two bounded extension and semantic items

10.1 L01: the Limit adapter silently chooses a side

Evidence and status. S/D/U. A helper-level semantic mismatch is identified. Inspected branch calls specify their side explicitly, so this report does not claim a current public wrong expansion.

The Mathics Limit adapter translates direction strings to the corresponding signed integer forms, but maps `Automatic` to `+1`, the from-below direction. The current official documentation describes the ordinary default as a real two-sided limit. These contracts differ [20, 36].

The simplest mathematical separator is

$$\lim_{u \uparrow 0} \frac{|u|}{u} = -1, \quad \lim_{u \downarrow 0} \frac{|u|}{u} = 1.$$

There is no common two-sided limit. A default mapped to `+1` therefore cannot serve as a general emulation of the current native default.

This is a latent issue because the inspected branch validator passes `Direction -> -1` in its positive local coordinate. That call is not affected by the default mismatch [11]. A safe repair is either to make the private adapter require an explicit side, or to implement a two-sided request by proving agreement of both admitted one-sided limits. It should not simply rename a one-sided result “Automatic.” Acceptance should include unequal sides, equal finite sides, and unresolved sides, while preserving the explicit branch controls.

This item should remain below the current branch/observable correctness obligations in priority. Its value is preventing a helper with broader-looking syntax from becoming an unsafe future dependency.

10.2 G01: terminating hypergeometric polynomials need no convergence gate

Evidence and status. S/D/E/U. The adapter gate and finite example are established. The interpreter may pre-evaluate a terminating example before the adapter sees it; no universal public failure is alleged.

The defining-Taylor helper admits a restricted monomial argument, real upper parameters, positive lower parameters, and $p \leq q + 1$ for a generalized ${}_pF_q$. That inequality supplies a nonzero convergence radius for the admitted nonterminating series. However, a nonpositive integer upper parameter can terminate the series even when $p > q + 1$; the resulting polynomial does not need that convergence condition [21, 37].

For example,

$${}_3F_1\left(\begin{matrix} -2, 3, 4 \\ 5 \end{matrix}; z\right) = 1 - \frac{24}{5}z + 8z^2. \quad (10)$$

The coefficient at degree one is $(-2)(3)(4)/5 = -24/5$. The degree-two coefficient is

$$\frac{(-2)_2(3)_2(4)_2}{(5)_2 2!} = \frac{2 \cdot 12 \cdot 20}{30 \cdot 2} = 8,$$

and all later terms vanish. At $z = 2u^3$ the polynomial becomes

$$1 - \frac{48}{5}u^3 + 32u^6.$$

Both identities are independently checked in the archive.

A narrow extension should detect exact terminating upper parameters *before* rejecting $p > q + 1$. It can retain the current positive-lower-parameter restriction initially, avoiding ambiguous denominator zeros and cancellation conventions. The smallest terminating index gives a finite coefficient budget. If the requested cutoff excludes part of the polynomial, the returned approximation still has a nonzero omitted tail; exactness is justified only when all nonzero terms have been retained or the remainder is otherwise proved zero.

The distinction between helper coverage and public behavior is important. A native simplifier might already reduce (10) to a polynomial, allowing the ordinary package path to succeed. In that case the public probe is a control, not evidence that the helper admits the original special-function syntax. The extension should be tested at its own admission boundary as well as through the public constructor.

This is an implementation opportunity within the existing finite-defining-series design, not an invitation to expand divergent nonterminating ${}_pF_q$ expressions as convergent Taylor series. No new asymptotic theorem is needed for the terminating polynomial.

11 API and development recommendations tied to the new findings

11.1 Make the failing contract visible, not merely the function name

A user seeing `ProductLog` in a result needs to know whether the runtime can preserve its symbolic meaning and whether it can evaluate the selected branch numerically. Those are different questions. Likewise, “unable to prove real domain” for a symbolic polynomial is not the same as “nonreal polynomial,” and “no neighborhood witness found” is not the same as “condition false near the endpoint.” The new findings justify making these distinctions in diagnostics at their actual failure sites.

This is a targeted extension of the already proposed capability schema, not a replacement schema project. For M01, the relevant capability is branch-aware exact conversion and numerical evaluation. For M02, it is a proof that coefficients are real under the retained assumptions. For M03, it is a positive radius together with the predicate proved on that radius. A small structured record at each boundary is more useful than a broad feature flag such as “Mathics supported.”

11.2 A dependency-ordered implementation plan

Stage	Work tied to this report	Required acceptance evidence
1	Characterize M01 exact, symbolic, and inexact routes before broadening the workaround. Correct the upstream diagnosis and retain branch metadata.	Official-kernel and Mathics controls; exact principal identity; branch--1 core value; intermediate conversion semantics.
2	Thread assumptions into polynomial-domain inference; add a constructive eventual certificate without changing fixed-radius statements.	Parameter/no-parameter controls; real/nonreal coefficients; tiny margins; crossing intervals; unknown-sign refusals.
3	Replace degree-dense coefficient extraction at the identified Mathics/Fourier consumers.	Equal coefficient/envelope results; support-fixed degree scaling; zero and cancellation controls; explicit resource refusals.
4	Reuse coherent source snapshots in portable validation and reject nonfinite deadlines.	Mutation fixtures, modular/standalone source identity, timeout parser controls, unchanged ordinary protocol behavior.
5	Resolve the private Limit default and add terminating hypergeometric admission.	Side-sensitive helper tests; finite polynomial identities; truncation/exactness and denominator controls.

This plan does not demote the earlier register's unresolved correctness obligations. In particular, an existing source-admission or branch-safety finding should not be postponed merely because a new Mathics feature is easier to demonstrate. The maintained register remains the owner of those older items [3, 4].

11.3 Testing the boundaries rather than inflating a function catalog

Each accepted repair should have a pair of tests: one showing the newly supported case, and one showing that the adjacent unsafe case is still refused or unresolved. A parameter-aware polynomial-domain helper needs an unproved-complex coefficient control. A constructive radius helper needs a fixed-interval crossing control. A terminating hypergeometric extension needs a nonterminating $p > q + 1$ refusal. A Lambert numeric bridge needs the wrong-branch control even when a fixed-point residual is small.

For performance, add a third dimension: vary the resource that the fix addresses while keeping unrelated structure fixed. The D-P06 benchmark should hold source weights and frequencies fixed while increasing logarithmic degree. Varying all three together cannot show that the dense-inner-polynomial bottleneck was removed. Record construction, first access, and repeated access separately where result caching or display can move work between phases.

The native-preservation comparison should remain part of every adapter change. Context-local design reduces the risk of changing official-Wolfram behavior, but the late definition rewrites and generated standalone order are still executable code. Source equality of selected definitions is useful evidence; it is not a substitute for the new behavior-specific regressions.

12 Reproducibility and supplied artifacts

12.1 What actually ran

The archive's `evidence/oracle-results.json` records **27 successful independent tests, zero failures, and zero errors**. The tests cover exact inverse coefficients, sign/radius certificates, sparse polynomial operations and extraction, hypergeometric termination, correct Lambert branch/argument interchange,

the A–B–A source countermodel, and finite timeout validation. The record includes 100 low-degree sparse-polynomial comparison cases and 400 polynomial-radius cases with three exact interior evaluations each. Those parameterized checks belong to the 27 named test methods; they are not additional native package tests.

The recorded reference run took approximately 0.85 seconds in this environment. That duration is only a reproducibility detail for the supplied reference tests. It is not a speed comparison with AsymptoticAnalysis, Mathics, or Wolfram Language. The separate `tooling-results.json` records **8 successful synthetic helper tests**: patch construction/idempotence/refusal and archive extraction/traversal/link controls.

Both records explicitly state that package tests, Mathics execution, and official-Wolfram execution were not performed. The raw console outputs are included. No checksum files are included; the reviewed Git revision and the source paths identify the review target.

12.2 Artifact map

Artifact	Purpose and status
<code>article/</code> <code>asymptotic-incremental-review.</code> <code>tex</code> and corresponding PDF	Complete article source and rendered document.
<code>code/reference_algorithms.py</code>	Executed independent rational/symbolic reference implementations; not a package port.
<code>code/run_oracles.py</code>	The 27-test independent oracle suite and JSON receipt generation.
<code>code/ReviewMathicsPrimitives.wl</code>	Unrun companion prototypes for sparse coefficient extraction, parameter-aware polynomial domains, and affine eventual certificates. No System or repository definitions are changed.
<code>code/portable_probe.wl</code>	Seven unrun characterization cases; output is preserved without guessing a pass/fail result.
<code>code/run_probes.py</code>	Unrun kernel-launch integration; obtains package inputs from a resolved Git archive and uses fresh processes. Archive utility tested separately.
<code>code/patch_timeout.py</code>	Tested synthetic patch generator; default is a printed diff, writing requires explicit <code>-apply</code> . Not applied to the repository.
<code>code/test_tooling.py</code>	Eight executed synthetic tests for the patch and archive helpers.
<code>evidence/novelty-ledger.csv</code>	Finding ancestry, scope, current evidence, and integration obligations.
<code>evidence/sources.json</code>	Pinned source-path inventory and relevant symbols.
<code>evidence/*results.json</code> and console records	Actual independent execution evidence, separated from unrun kernel probes.

12.3 Commands

From the archive root, use an environment containing SymPy 1.14.0 and mpmath 1.3.0 for the independent suite:

```
python code/run_oracles.py
python code/test_tooling.py
```


The first script reproduces its evidence file and console diagnostics. These commands do not install or execute Mathics.

For a local checkout containing the reviewed commit, the following commands characterize the pinned source using an installed interpreter:

```
python code/run_probes.py --repo /path/to/Asymptotic \
  --mathics-python /path/to/mathics-env/bin/python \
  --entry modular --output evidence/mathics-characterization.json

python code/run_probes.py --repo /path/to/Asymptotic \
  --wolfram /path/to/WolframKernel \
  --entry standalone --output evidence/wolfram-characterization.json
```

Use `-case` to select individual cases and `-timeout` to set a finite positive process deadline. The default revision is the title-page commit. A different `-commit` intentionally changes the review target and must be reported as such. The runner loads committed archive contents, not uncommitted local edits.

The Wolfram Language companion can be loaded *after* the package in a disposable kernel for candidate-helper testing. It is not automatically installed by the probes. Its mathematical design is supported by the reference work, but its WL/Mathics syntax and evaluator behavior remain an integration obligation.

To rebuild this document:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error \
  asymptotic-incremental-review.tex
pdflatex -interaction=nonstopmode -halt-on-error \
  asymptotic-incremental-review.tex
```

A standard TeX installation needs the packages named in the preamble. No proprietary fonts or shell-escape tooling are required.

13 Conclusions

AsymptoticAnalysis is a substantial mathematical software project with more structure than a front end to native expansion commands. Its strongest design choices are explicit scales, complete coefficient blocks, retained real-branch information, conservative remainder handling, and a separation between formal residuals, numerical evidence, and interval certificates. Wolfram Language nevertheless supplies broader native asymptotic facilities; a fair comparison treats those as real competitors and interoperability targets, not as absent functionality [6, 33, 34, 35].

The inspected Mathics effort is serious and appropriately isolated. It now has dedicated adapters, tests, documentation, and CI. Full compatibility remains a goal rather than an established result. In particular, exact expression interchange, parameter-aware proof interfaces, and numerical branch evaluation require separate evidence. A successful load or an unchanged symbolic printout does not prove those contracts [13, 27, 18].

The most actionable additions are therefore narrow: repair the ProductLog interchange boundary; pass retained assumptions into polynomial-domain inference; construct a real neighborhood witness instead of trying seven absolute scales; keep coefficient polynomials sparse in their own grading; and make test-source acquisition match the claim made by its receipt. The Limit-default and terminating-hypergeometric items are useful bounded follow-ons.

The supplied article and artifacts provide exact mathematical acceptance targets and reproducible independent checks for these changes. They do not claim that the proposed WL helpers are merged, that the public probes have passed, or that any old ledger item is closed by unrelated tests. That separation is necessary for the next implementation step to improve both capability and trustworthiness.

A Novelty crosswalk and evidence boundaries

ID	Nearest earlier or documented item	Actual increment and boundary
M01	D05; documented principal-only Mathics workaround	Exact SymPy route, inherited numeric arity, two-direction conversion semantics, and user-source/exact-core exposure. No current Mathics output claimed.
M02	C05 assumption retention; bounded polynomial branch plan	A new adapter's signature discards the opportunity to use retained parameter facts. Polynomial real domain and monotonicity kept separate.
M03	Bounded affine eventual-condition plan	Seven predetermined radii are not scale-independent admission. Constructive radius theorem, parameter scope, and fixed-interval non-substitution rule supplied.
D-P06	P06/P08; not C01 or W3-04	A separate inner-logarithmic-degree allocation path in Mathics and common Fourier code. Sparse reference, not native speedup claim.
D-V01	V01/V02; current snapshot-based parity workflow	Endpoint equality versus execution-time source identity, with A-B-A countermodel. Reuse the existing snapshot approach.
L01	Direction/API consistency themes	Specific new private default mapping differs from the current native default. Latent because inspected consumers pass a side.
G01	Broader special-function coverage goals	Safe finite-termination admission before the nonterminating convergence gate. Public native pre-evaluation may bypass the gap.
T01	Runner resource-validation themes	Exact NaN/infinity acceptance mechanism and a tested finite-positive patch generator. No ordinary timeout regression alleged.

B Public characterization catalog

Probe ID	Mathematical target	Do not infer
<code>lambert-source</code>	Both forms describe $1 + x$.	That held entry prevents all later native evaluation.
<code>lambert-core</code>	$-W_{-1}(-1/y)$, with value 2 at $y = e^2/2$.	That an inert symbolic expression supplies numerical compatibility.
<code>parameter-domain</code>	The real increasing inverse of $ax + x^3$ under $a > 0$, with coefficients in (7).	That inverse-callable syntax takes the same route on both runtimes.
<code>narrow-neighborhood</code>	The identity germ under either positive upper cutoff.	That failure at a chosen radius disproves eventual truth.
<code>sparse-small</code>	Two coefficient rules for $1 + q^{1000}$.	That a small output proves a sparse intermediate algorithm.

Probe ID	Mathematical target	Do not infer
<code>limit-default</code>	Left and right limits disagree; explicit sides remain distinct.	That the helper mismatch currently corrupts public branch calls.
<code>terminating-hypergeometric</code>	$1 - 24x/5 + 8x^2$.	That public success establishes admission by the defining-Taylor helper.

C Source inspection inventory

The citations below identify the exact public sources used. For large modules, inspection was focused on the named functions and relevant ranges rather than a claim of exhaustive line-by-line verification of the repository. The machine-readable inventory in the archive mirrors the important paths.

The main implementation audit covered public constructor contracts, result accessors, assumption capture, inverse coefficient/residual interfaces, the Fourier parser/coefficient algebra, Gamma inverse valuation consumers, exact exponential-core construction, inverse-callable routing and branch validation, and the Mathics bootstrap/adapters. The validation audit covered the complete portable Python runner, the Mathics workflow, and the documented snapshot/definition-preservation process. The historical exclusion audit used the maintained three-wave crosswalk and the original review-six ledger.

References

- [1] Vladimir Reshetnikov, *Asymptotic repository, reviewed commit*. [Commit 513917b5b387152256b14ac76d92dd30cd13a11d](#). September 10, 2026 UTC.
- [2] [external-reports/code-review/README.md](#). Three-wave review index and attributed finding counts.
- [3] [docs/development/CODE_REVIEW_STATUS.md](#). Implementation register, wave-one/two crosswalk, and existing correctness, resource, API, and extension items.
- [4] [docs/development/WAVE_3_INTAKE.md](#). Complete wave-three finding crosswalk and implementation scope.
- [5] [external-reports/code-review/wave-1/code-review-6/evidence/findings.csv](#). Original review-six ledger, particularly A06, A07–A10, and A16.
- [6] [src/Kernel/AsymptoticAnalysis.wl](#). Public usages and bootstrap; assumption boundary; exactRealQ; result accessors; inverse/model interfaces.
- [7] [src/Kernel/FourierCoefficients.wl](#). `fourierPoly`, `fourierEnvelope`, `coefficient/jet` parsing, convolution, and inverse construction.
- [8] [src/Kernel/GammaInverse.wl](#). `gammaInverseModel`, `gammaInverseRows`, `gammaInverseBound`, and construction.
- [9] [src/Kernel/ExponentialCorePerturbation.wl](#). Coordinate selection, `exponentialCoreExactInverse`, and exact-core sector construction.
- [10] [src/Kernel/LambertInverse.wl](#). Leading logarithmic/exponential models and `lambertConstruct`.
- [11] [src/Kernel/InverseFunctionBranches.wl](#). `inverseFunctionSelectBranchInternal`, domain admission, monotonicity, and deleted-neighborhood validation.
- [12] [src/Kernel/InverseFunctionExpressions.wl](#). `inverseFunctionEventually`, `forwardPublic`, and callable inverse routing.
- [13] [docs/Mathics/COMPATIBILITY.md](#). Current target scope, limitations, historical acceptance records, and snapshot-based native definition comparison.
- [14] [docs/Mathics/CALLABLES.md](#). Named functions, held extraction, and bounded inverse-branch facts.
- [15] [src/Kernel/MathicsCompatibility.wl](#). Context isolation, `Module/Return`, association helpers, inert System names, and loading boundary.
- [16] [src/Kernel/MathicsAssumptions.wl](#). Conservative realness, signs, relations, and assumption consequences.

- [17] [src/Kernel/MathicsInverseBranches.wl](#). Polynomial-domain adapter, affine interval proofs, and seven-radius eventual-condition schedule.
- [18] [src/Kernel/MathicsCoreFunctions.wl](#). Principal ProductLog normalization at selected construction sites and deadline preservation.
- [19] [src/Kernel/MathicsAlgebra.wl](#). Univariate CoefficientRules adapter.
- [20] [src/Kernel/MathicsSimplification.wl](#). Guarded native simplification, Series assumptions, and Limit direction translation.
- [21] [src/Kernel/MathicsTaylor.wl](#). Restricted defining Taylor provider and generalized-hypergeometric admission.
- [22] [src/Kernel/MathicsSpecialFunctions.wl](#). Finite Stirling LogGamma model and explicit Poincare remainder.
- [23] [src/Kernel/MathicsCalls.wl](#). Held Extract adapter.
- [24] [src/Kernel/MathicsLists.wl](#). Private-reference empty-list Map workaround.
- [25] [src/Kernel/MathicsFormatting.wl](#). Arithmetic UpValues and OutputForm installation.
- [26] [validation/run_mathics_tests.py](#). Portable per-case subprocess runner, frozen test script, source observations, protocol, and timeout input validation.
- [27] [.github/workflows/mathics.yml](#). Modular/standalone CI matrix and grouped portable regressions.
- [28] Mathics3 developers, [mathics/builtin/specialfns/expintegral.py](#), tag 10.0.1. ProductLog and LambertW declarations.
- [29] Mathics3 developers, [mathics/core/builtin.py](#), tag 10.0.1. SympyFunction conversion and MPMathFunction arity/evaluation paths.
- [30] Wolfram Research, *Series*. <https://reference.wolfram.com/language/ref/Series.html>. Documentation consulted September 9, 2026.
- [31] Wolfram Research, *SeriesData*. <https://reference.wolfram.com/language/ref/SeriesData.html>. Documentation consulted September 9, 2026.
- [32] Wolfram Research, *InverseSeries*. <https://reference.wolfram.com/language/ref/InverseSeries.html>. Documentation consulted September 9, 2026.
- [33] Wolfram Research, *Asymptotic*. <https://reference.wolfram.com/language/ref/Asymptotic.html>. Documentation consulted September 9, 2026.
- [34] Wolfram Research, *AsymptoticSolve*. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>. Documentation consulted September 9, 2026.
- [35] Wolfram Research, *DiscreteAsymptotic*. <https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html>. Documentation consulted September 9, 2026.
- [36] Wolfram Research, *Limit*. <https://reference.wolfram.com/language/ref/Limit.html>. Current default direction and one-sided conventions; consulted September 9, 2026.
- [37] NIST Digital Library of Mathematical Functions, *Section 16.2: Definition and Analytic Properties*. <https://dlmf.nist.gov/16.2>, especially the terminating case for $p > q + 1$. Consulted September 9, 2026.